

ClaimsForge

The Trust Layer for AI Claims Resolution

Multi-agent Gemini AI that resolves e-commerce damage claims — and proves it wasn't fooled.

Gemini 2.5 Flash + Vision + Embedding

7 Specialist Agents

AWS-IAM-style Supervisor

Stripe-Radar Trust Score

100% E2E (20/20)

MIT · Vultr

LIVE DEMO

<http://45.32.154.255>

REPO

github.com/zhenyueD/claimsforge

E-commerce claims are slow, costly, and now actively spoofed by AI.

\$743B

Annual returns
US e-commerce, 2025
Source: National Retail Federation

3 days

Mean human resolution time
per low-value damage claim
Industry benchmark, 2025

99%

Insurers have seen
AI-tampered evidence
Verisk 2026 State of Insurance Fraud

32%

Feel confident
catching deepfakes
Verisk 2026 · same study

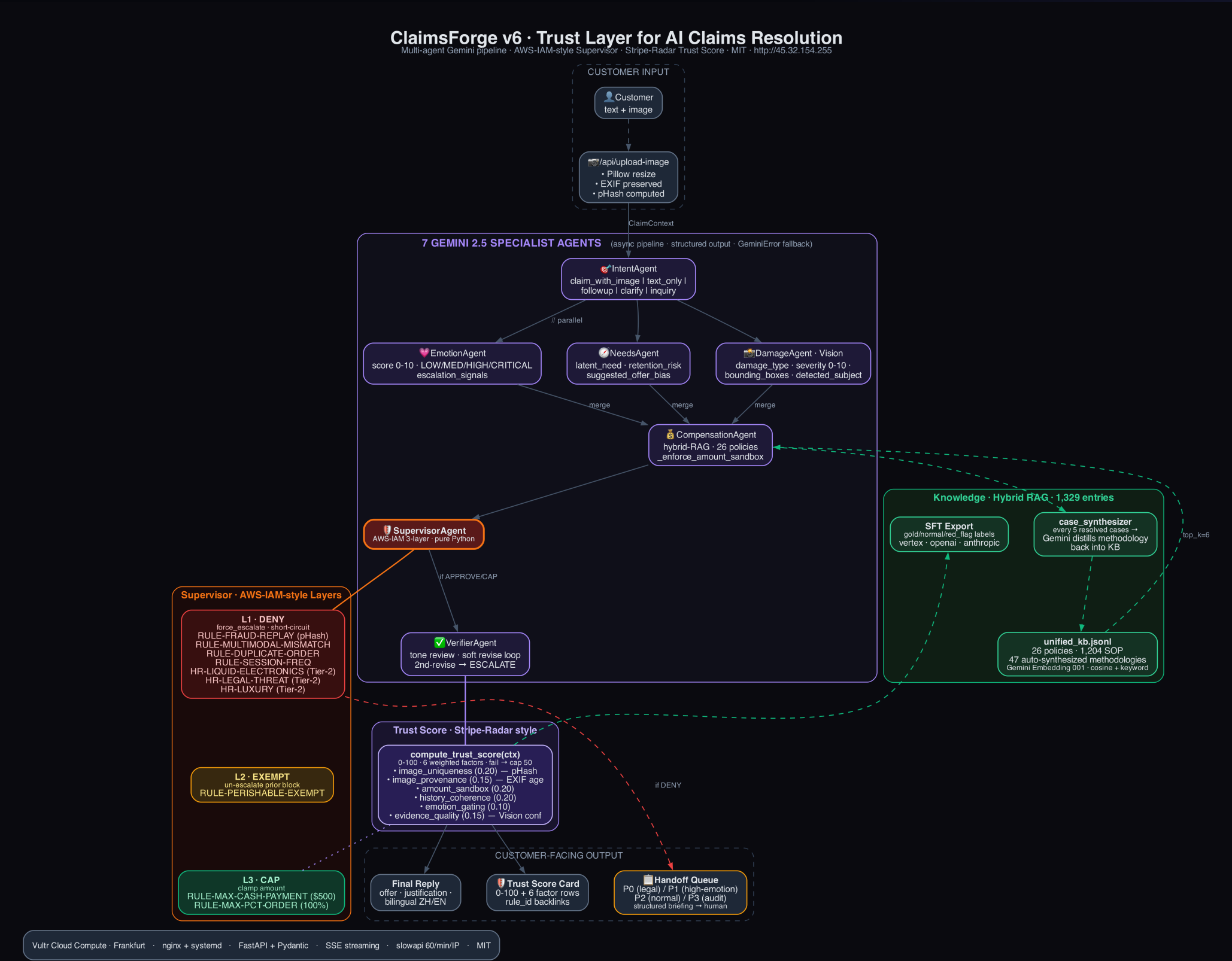
Existing AI agents automate the dialogue. None of them prove the AI didn't get fooled.

Sierra, Decagon, Intercom Fin — all impressive at generating empathetic replies. But finance, compliance, and merchant ops teams keep asking the same question: "How do I know the AI just paid the right amount to the right claim?"

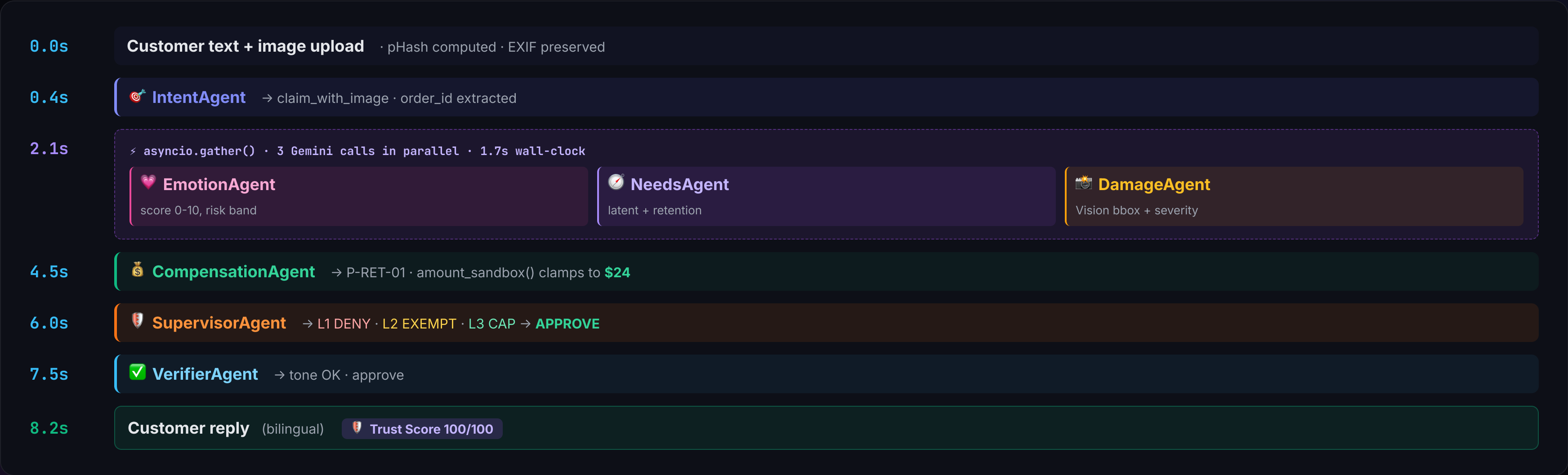
Sierra & Decagon decide **what the AI says**. ClaimsForge also **proves it wasn't fooled**.

CAPABILITY	SIERRA / DECAGON / FIN	CLAIMSFORGE V6
Multimodal damage adjudication	✗ Text + voice, no vision	✓ Gemini 2.5 Vision · bbox · 0-10 severity
Hard-rule safety supervisor	⚠ Model-of-models cross-check (opaque)	✓ Pure-Python · AWS-IAM 3-layer · auditable
Deepfake / fraud detection	✗ Not exposed publicly	✓ pHash + EXIF age + multimodal consistency
Customer-facing Trust Score	✗	✓ Stripe-Radar style · 0-100 · 6 factors · rule_id links
Authoring SOPs	Customer must author AOP upfront	✓ Cases → auto-distill methodology (89 today)
Hot-editable rules (no deploy)	Configured by Sierra team	✓ Tier-2 JSON DSL + admin UI toggle
Deployment	Managed service · \$200K–2M ARR · 3–6mo PoC	✓ MIT · 1 Vultr VM · 30-min self-deploy
Fine-tune data export	Black box	✓ Auto-labeled gold/red_flag · Vertex/OpenAI/Anthropic

7 Gemini agents · 3-layer supervisor · self-evolving knowledge



One claim · 7 agents · 6 Gemini calls · async parallel



Parallel fan-out

Emotion + Needs + Damage run in **one** `asyncio.gather()` — three Gemini calls × ~1.5s = **1.7s wall-clock** instead of 5s sequential.

Deterministic in the middle

Supervisor + amount_sandbox are **pure Python** — sub-millisecond. LLM cannot talk its way past a hard rule.

SSE-streamed to UI

Each agent's trace ships the instant it completes — the customer watches the pipeline live, no "thinking..." spinner anxiety.

7 Gemini 2.5 specialists

async fan-out · structured output · GeminiError fallback at every node



IntentAgent

Routes claim_with_image / text_only / followup / clarify / inquiry. Extracts order_id.

flash · function-calling



EmotionAgent

0-10 score · LOW/MED/HIGH/CRITICAL risk · extracts escalation_signals (lawyer / 12315 / media).

flash · 7-trigger schema



NeedsAgent

Surface vs latent need · retention_risk · suggested_offer_bias (refund / replacement / credit).

flash · biases offer



DamageAgent

Vision. damage_type · severity 0-10 · normalized bounding_boxes · detected_subject (for multimodal gate).

flash · vision · bbox



CompensationAgent

26 policies · hybrid-RAG over 1,329 KB entries · Python amount sandbox clamps LLM dollar values.

flash + embedding



SupervisorAgent

The differentiator. Pure-Python, AWS-IAM 3 layers: DENY → EXEMPT → CAP. Tier-1 code + Tier-2 hot JSON.

deterministic · auditable



VerifierAgent

Soft tone + fit review. Max 1 revise. 2nd-revise auto-escalates (never accept unverified offer).

flash · safety net

⚡ Why this beats LangGraph / CrewAI

Zero framework. One orchestrator.py · 522 lines · pure asyncio.gather + typed ClaimContext. Emotion || Needs || Damage run in parallel — p50 latency 8.9s end-to-end, 6+ Gemini calls per claim.

AWS-IAM-style Supervisor — explicit Deny / Exempt / Cap in pure Python

🚫 L1 · DENY — force_escalate · short-circuit

RULE-FRAUD-REPLAY (pHash)

RULE-MULTIMODAL-MISMATCH

RULE-DUPLICATE-ORDER

RULE-SESSION-FREQ

HR-LIQUID-ELECTRONICS · Tier-2

HR-LEGAL-THREAT · Tier-2

HR-LUXURY · Tier-2

↺ L2 · EXEMPT — un-escalate prior block

RULE-PERISHABLE-EXEMPT — food / cosmetic claims skip the no-image rule

⚠️ L3 · CAP — clamp without escalating

RULE-MAX-CASH-PAYMENT · \$500

RULE-MAX-PCT-ORDER · 100%

Why IAM, not LLM judgement?

LLMs hallucinate policies and overrule cash caps when the customer writing is empathetic. A pure-Python gate **cannot** be talked out of a hard rule. Every decision logs the `rule_id` that fired.

3-Tier Rule Authority

Tier 1 — code constitution, can't toggle at runtime

Tier 2 — `data/hard_rules.json`, business toggle via admin UI

Tier 3 — KB embeddings, soft guidance only

```
# Tier-2 DSL — strict operator whitelist · no dynamic exec
{
  "id": "HR-LUXURY",
  "when": { "product_hint_in": ["luxury", "jewelry", "watch"] },
  "verdict": "force_escalate",
  "reason": "Luxury manual handling regardless of policy",
  "owner": "claims-ops@company.com",
  "active": true
}
```

Stripe-Radar-style Trust Score on every offer — 0-100 · 6 weighted factors · rule_id traceable

The factor breakdown

image_uniqueness	0.20	pHash collision against approved-image anchor set
image_provenance	0.15	EXIF DateTimeOriginal age (≥365d → fail)
amount_sandbox	0.20	Python clamp on LLM dollar output
history_coherence	0.20	Duplicate-order detection + session frequency
emotion_gating	0.10	Legal/regulator signals · score 0-10
evidence_quality	0.15	DamageAgent confidence + multimodal mismatch

Weighted sum × 100. Any fail factor caps the total at 50 — the "do not trust" signal the UI surfaces in red.

In the customer-facing UI

Trust Score · IAM deny

Layer=deny

50/100

✖ image_uniqueness · pHash collision · RULE-FRAUD-REPLAY

✔ amount_sandbox · LLM within bounds

✔ history_coherence · no duplicate order

✔ emotion_gating · score 3.0/10

✔ evidence_quality · damage conf 90%

⚠ image_provenance · photo taken 853 days ago

Each factor carries a rule_id backlink so a finance auditor can jump from "Trust 50" all the way down to the supervisor rule that fired and the JSON predicate that matched.

Clean claim → 8-second auto-refund · Trust Score 96

- 1

Customer: "My mug arrived cracked, order ORD-MUG-001" + mug_crack.jpg
- 2

IntentAgent · claim_with_image · order_id extracted · 0.4s
- 3

Emotion || Needs || Damage in parallel · 2.1s.
Damage: crack · severity 8/10 · conf 0.90 · bbox painted on image
- 4

CompensationAgent · hybrid-RAG retrieves 6 KB entries · selects P-RET-01 · proposes full_refund \$24
- 5

Supervisor · all 7 deny rules pass · all 6 trust factors green · approve
- 6

VerifierAgent · tone OK · approve
- 7

Customer reply · bilingual (auto-detect) · 8.2s end-to-end

What the customer sees

"That crack along the rim makes the mug unusable — I've issued a full refund of \$24 to your original payment method. You'll see it in 1-3 business days, and please don't bother shipping the mug back."

Trust

100

/100

layer=approve

✓

image_uniqueness · no pHash collision in 47 anchors

✓

image_provenance · photo taken 2 days ago

✓

amount_sandbox · LLM within policy

✓

history_coherence · no duplicate

✓

emotion_gating · score 3.0/10

✓

evidence_quality · damage conf 90% · crack

Three buttons under every reply:

✓ Accept

 ·

↔ Reject — renegotiate

 ·

👤 Talk to human

 . Accept → labels case gold for SFT export.

Three adversarial attacks, three deterministic denies

Attack 1 · pHash Replay

Customer submits a damage photo. **Auto-refund approved.**

Attacker re-submits **same photo** from new session, new email.

```
layer = deny
matched = [RULE-FRAUD-REPLAY]
reason: pHash collision
Hamming=0 cross-session
```

Pure Python · <5ms · zero LLM cost

Attack 2 · Deepfake / Stale Photo

Customer uploads an "evidence" photo. Looks legit to LLM Vision.

EXIF DateTimeOriginal: 2024-01-15. Order was placed last week.

```
factor = image_provenance
status = fail
photo taken 853 days ago
→ Trust cap at 50
```

99% of insurers have seen this · 32% catch it (Verisk 2026)

Attack 3 · Text/Image Mismatch

Customer text: *"My ceramic mug arrived cracked"*

Image actually shows: a smartphone.

```
layer = deny
matched = [RULE-MULTIMODAL-MISMATCH]
hinted: "ceramic mug"
detected: "smartphone"
```

Sierra-impossible — they have no vision channel to cross-check

E2E validation

20/20 = 100% joint accuracy on adversarial test set including all three attacks above + prompt injection + amount overclaim + emotion spoofing + multi-turn inconsistency + duplicate-order. **p50 latency 8.9s** end-to-end.

Duplicate refund blocked · ops toggle rules live

Duplicate-order attack

- T1

Customer: "Mug cracked, ORD-MUG-T03" → bot refunds \$24 → customer clicks

Accept

 ✓
- T2

Same customer 5 min later: "Mug cracked, ORD-MUG-T03 — please refund"
- T3

Supervisor ·

RULE-DUPLICATE-ORDER

 scans history →

resolve:accept

 already exists for this order_id →

force_escalate

```
{
  "layer": "deny",
  "matched_rules": ["RULE-DUPLICATE-ORDER"],
  "reasons": [
    "Duplicate claim on ORD-MUG-T03",
    "prior refund already accepted"
  ]
}
```

Pure-Python history scan. Cost: 0 LLM calls. Saves merchant from being double-charged on every claim history rewind.

Tier-2 hot-rule toggle

Ops team needs to disable the **HR-LUXURY** rule for a flash sale: every luxury claim should auto-pay this weekend. **No code deploy.**

/admin · Tier-2 Hard Rules

HR-LIQUID-ELECTRONICS

Liquid damage on electronics requires manual warranty

● ENABLED

HR-LEGAL-THREAT

Legal / regulator signals require human

● ENABLED

HR-LUXURY

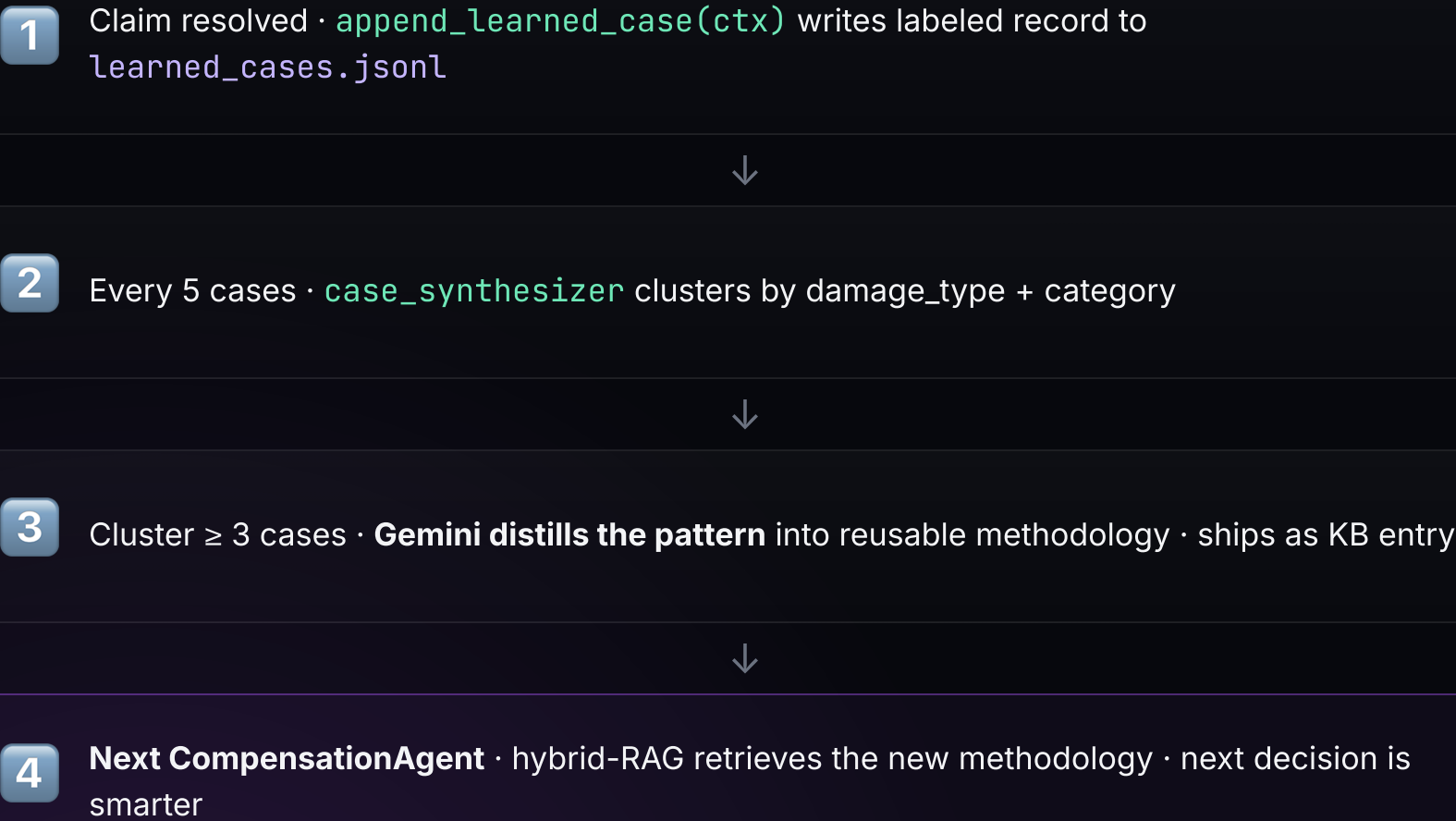
Luxury category manual handling

○ DISABLED

Click the switch · atomic temp+rename write to **hard_rules.json** · supervisor mtime-cache invalidates within 60s · next claim sees the new rule set.

Cases → auto-distilled SOPs. No human SOP authoring required.

The closed-loop



Sample auto-synthesized methodology

`methodology` · `derived_from_8_cases` · `quality=0.84`

Cracked Mug Damage Resolution — auto-distilled

WHEN: Customer reports a cracked ceramic mug ≤ \$30 with photo evidence, low emotion (≤5/10).

DO: Issue `full_refund` equal to order value · do not require return · explicitly name the visible damage location in reply · ship in customer's language.

89

methodologies
auto-synthesized

1,329

total KB entries
policies + SOP + methodologies

100%

embedded
gemini-embedding-001

Browse live at </methodologies> — full text, source case count, quality score, search.

Every approved claim is **labeled training data** — one-click fine-tune export

Auto-labeled at resolution time

gold supervisor approved · customer accepted (best signal)

normal supervisor approved · no explicit accept yet

red_flag escalated / supervisor blocked (anti-examples)

/api/admin/finetune-dataset

```
# Vertex AI Gemini fine-tuning format
GET /api/admin/finetune-dataset?
  format=vertex&quality=gold

# Same endpoint, OpenAI format
GET /api/admin/finetune-dataset?
  format=openai&quality=gold

# Same endpoint, Anthropic format
GET /api/admin/finetune-dataset?
  format=anthropic&quality=gold
```

Response: streaming NDJSON · headers carry X-Sft-Rows.

Why this matters commercially

At **~2,000 gold cases**, the merchant runs **one** Vertex AI / OpenAI tuning job that produces a smaller, faster, ClaimsForge-shaped model.

~30%

prompt token reduction
no 26-policy preamble

~50%

per-call latency reduction
smaller model + smaller context

Live admin UI

 SFT Dataset Export · data flywheel

Format

vertex (Gemini) ▾

Quality

gold ▾

 Download .jsonl

This is **SFT** (Supervised Fine-Tuning), not RLAIF. RLAIF requires a reward model + RL trainer — out of scope. We do the supervised pairs export honestly.

Built for SMB merchants Sierra can't reach.

90%

Auto-resolution rate target
for low-value damage claims

edge cases route to handoff queue with structured
P0-P3 briefing

~8s

End-to-end resolution
vs. 3 days human back-and-forth
p50 of 7-agent async pipeline · 6+ Gemini calls

30min

Self-deploy from scratch
vs. Sierra 3-6mo managed PoC
single Vultr VM · nginx + systemd · MIT

\$0

License fee
vs. Sierra \$200K-2M ARR
MIT · merchant pays only Gemini + Vultr

What the merchant gets day 1

- 26-policy DSL · 1,329 KB entries · 89 auto-synthesized methodologies
- Customer-facing Trust Score on every offer — built-in compliance artifact
- P0-P3 handoff queue with structured briefing (Reason · Action · Bot proposal)
- Admin dashboard · Operations metrics · KB browser · Methodology showcase
- SSE-streaming UI · in-flight agent traces visible to customer

Where ClaimsForge wins

- **Shopify / WooCommerce / Magento SMBs** — too small for Sierra, too painful manually
- **Insurance MGAs** — need deepfake screening + auditable decisions for regulators
- **Marketplace operators** — high-volume low-value claims with fraud surface
- **Cross-border e-commerce (CN / EN)** — native bilingual, no translation layer
- **SaaS subscription cancels** — Trust Score + emotion gating = retention saves

TRY IT NOW

ClaimsForge

The Trust Layer for AI Claims Resolution — built for the deepfake era.

LIVE DEMO

<http://45.32.154.255>

Pre-loaded with 5 demo scenarios:
Cracked mug · Scratched laptop · Torn jacket ·
Angry customer + legal threat · Vague claim

SOURCE · MIT

github.com/zhenyueD/claimsforge

👉 30-SECOND JUDGE TEST

Open the demo · click 🤨 Angry customer + legal threat scenario · watch RULE-LEGAL-THREAT fire in the Supervisor trace and Trust Score drop to 50. That's the whole pitch — go check.

Gemini 2.5 Flash + Vision + Embedding

FastAPI · Pydantic · asyncio

AWS-IAM Supervisor

Stripe-Radar Trust Score

pHash · EXIF Provenance

Hybrid-RAG · 1329 entries

SFT Export · Vertex/OpenAI/Anthropic

Built by **zhenyueD** · Milan AI Week 2026 / AI Agent Olympics · Powered by **Google Gemini** + **Vultr Cloud Compute (Frankfurt)**

Numbers as of 2026-05-17 build · live demo counts may exceed (KB grows on every claim)